



Architecture Blueprint for Your iQOO AI Developer & Team Control Plane

Architecture verdict

Yes — **the product you described is technically possible with current infrastructure**, and you do **not** need to make the iQOO phone itself run the coding environment, Git repository, production logs, CI/CD, or cloud credentials.

The clean architecture is:

The iQOO phone is the human control plane.
Supabase is the shared team/state layer.
Cloudflare is the API and event-ingestion layer.
An orchestrator is the brain coordinating tasks.
NVIDIA NIM or another large model is the heavy reasoning engine.
Vercel Sandbox is the disposable cloud computer where code actually runs.
GitHub is the source of truth for code.
Vercel/AWS/Cloudflare are observed and controlled through narrowly scoped integrations.

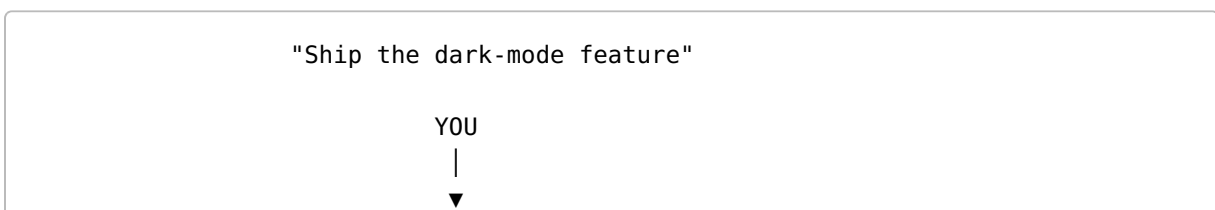
That division is especially appropriate for the iQOO hackathon because the official event describes itself as phone-first, alternates between periods where both devices are available and periods where work is phone-only through Office Kit, and explicitly gives extra weight to local/open-source AI being at the core. ¹

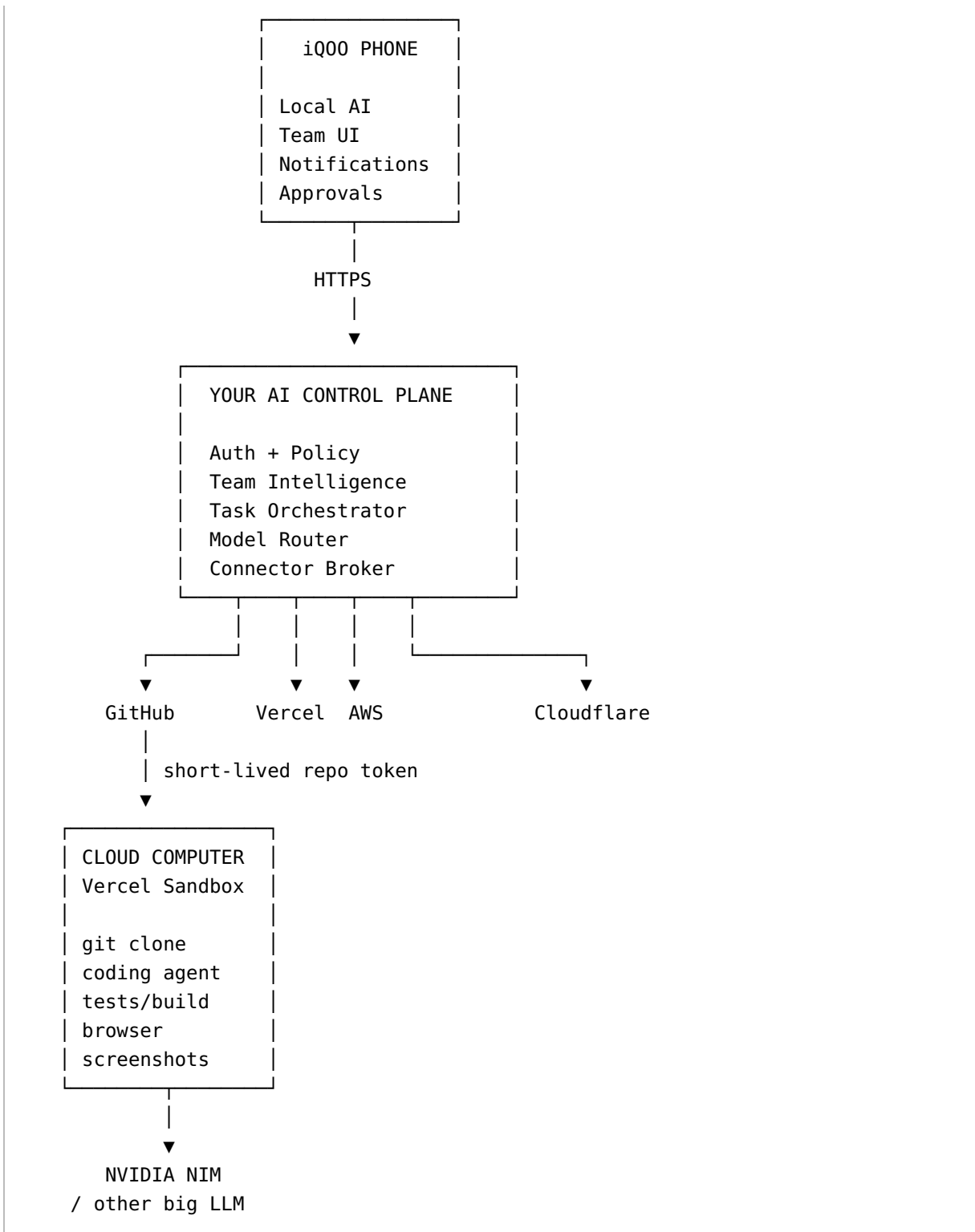
The biggest architecture correction I would make to your original concept is this:

Do not create “one API key that controls GitHub + AWS + Vercel + Cloudflare.”

Instead, create **one application identity** that the iQOO phone uses to talk to *your* control plane. Behind it, every provider gets its own separately scoped credential. GitHub Apps can mint installation tokens restricted to repositories and permissions, and those tokens expire after one hour; AWS STS can similarly issue temporary role credentials. Cloudflare supports scoped API tokens and server-side secrets. ²

So from the user's point of view there is one login and one AI:





The important insight is that **the model is not your backend**. The model proposes and reasons. Your orchestrator owns state and permissions. The sandbox executes code. Provider connectors perform privileged operations.

That difference will make your project much more believable to judges.

What should sit where

Here is the exact placement I would recommend for the system you described.

Layer	What sits here	Your features handled here
iQOO Android app	UI, voice/chat, small local model, notifications, approvals, team presence display, dashboard	8, 11, 20, 21, 22, 25
Supabase	Authentication, teams, projects, roles, tasks, incidents, deployment metadata, realtime team state	15, 16, 17, 21, 22, 23
Cloudflare Worker	Public API gateway, provider webhook receiver, auth verification, rate limiting, event normalization	7, 10 and ingress for 2–9
Cloudflare Queues / Workflow	Asynchronous events and durable multi-step jobs	3–6, 18, 19, 24
AI Orchestrator	Planner, task router, ownership resolver, policy engine, model router, connector broker	10, 12, 14, 18, 19, 23, 24
NVIDIA NIM / cloud models	Hard reasoning, repository analysis, debugging, code generation	1, 12
Vercel Sandbox	The “mini cloud computer”: clone repo, edit files, run terminal/tests/build/app/browser	1, 13
GitHub App	Repository access, branch/commit/PR operations, repository events	1, 2
Vercel connector	Preview/production deployment information, build state and deployment control	3, 4, 5
AWS connector	CloudWatch logs, infrastructure state, costs through Cost Explorer	5, 6, 9
Cloudflare connector	CDN/Worker infrastructure information, optional deployment/control actions	5, 6, 7
Firebase Cloud Messaging	Wake the Android app for incidents/tasks when it is not actively connected	20
Android Keystore	Device-bound signing key for high-risk approvals	24, 25

This is not just theoretical. Vercel's current Sandbox product explicitly supports executing untrusted or agent-generated code in isolated Linux microVMs, gives the sandbox its own filesystem and network, supports file editing, commands and live development servers, and exposes SDKs for creating and controlling the environment programmatically. Vercel specifically positions it for agent workflows and coding agents. ³

The iQOO phone

The phone should **not** be a tiny copy of GitHub Desktop.

It should be the place where humans interact with the autonomous system.

I would put these on-device:

iQOO APP

Home

- └─ Project health
- └─ Production status
- └─ Active teammates
- └─ Active AI agents
- └─ Open incidents
- └─ Today's cloud cost

Agent

- └─ Voice command
- └─ Text command
- └─ Camera input
- └─ Task plan
- └─ Agent progress

Team

- | | | |
|-------------|----------|-----------|
| └─ Sid | Frontend | Working |
| └─ Arjun | Backend | Available |
| └─ Ravi | DevOps | Working |
| └─ AI Coder | Agent | Running |

Approvals

- └─ Merge PR
- └─ Production deploy
- └─ Rollback
- └─ Infrastructure change

Incident

- └─ Error
- └─ suspected cause
- └─ owner
- └─ relevant commit
- └─ [Take it] [Delegate to AI]

Your local model should perform **small, latency-sensitive, privacy-friendly jobs**, not repository-wide coding.

For example:

```
"Something is wrong with checkout,  
find it and fix it"
```

can become locally:

```
{  
  "intent": "investigate_incident",  
  "project": "storefront",  
  "scope": "checkout",  
  "action_requested": "investigate_and_fix"  
}
```

Then the cloud does the expensive investigation.

Current Android tooling supports custom on-device models through LiteRT, while Qualcomm AI Hub currently provides tools for optimizing and deploying models to Qualcomm hardware; Qualcomm's GenieX provides a unified runtime across NPU, GPU and CPU. Whether you can use the NPU path specifically depends on the exact iQOO hardware loaned at the hackathon, so I would design a CPU/GPU fallback rather than assuming a particular Snapdragon configuration. ⁴

That gives you a good hackathon story:

Local AI decides what you want. Cloud AI figures out how to do it. The cloud computer executes it. Your phone remains the authority.

Supabase

Supabase becomes your **shared organizational memory**.

It should know:

```
Users  
Teams  
Projects  
Team members  
Developer roles  
Repositories  
Tasks  
Incidents  
Deployments  
Agent runs  
Human approvals
```

Audit events
Integration metadata
Cost snapshots
Notification devices

I would start with something approximately like:

```
users
  id
  name
  avatar

teams
  id
  name

team_members
  team_id
  user_id
  role
  permissions

projects
  id
  team_id
  name
  environment

project_integrations
  project_id
  provider
  provider_account_id
  secret_reference

repositories
  project_id
  github_installation_id
  repo_owner
  repo_name

tasks
  id
  project_id
  title
  type
  assigned_user_id
```

```
    assigned_agent_id
    status
    priority

incidents
  id
  project_id
  severity
  source
  owner_user_id
  status
  suspected_commit

deployments
  id
  project_id
  commit_sha
  environment
  status
  preview_url

agent_runs
  id
  task_id
  sandbox_id
  model
  status

approvals
  id
  action
  requested_by
  approved_by
  signature
  status

audit_events
  id
  actor_type
  actor_id
  action
  resource
  timestamp
```

Supabase Realtime Presence is a particularly good fit for things like **online, available, currently working on task X, viewing project Y** because Supabase explicitly describes Presence as shared small pieces of

state and says it is best for slow-changing states such as online/offline or current-page activity. Supabase advises using Broadcast rather than Presence for high-frequency fire-and-forget events. ⁵

So don't send this through Presence:

```
Sid typing cursor X = 871
Sid typing cursor X = 872
Sid typing cursor X = 874
...
```

Do send this:

```
{
  "user_id": "sid",
  "project_id": "bimax",
  "status": "working",
  "task_id": "incident_72",
  "activity": "frontend_fix"
}
```

The persistent truth — tasks, incidents, deployment state, assignments — belongs in Postgres. Presence is only the ephemeral “**what is everyone doing right now?**” layer.

And you should enforce team isolation with database authorization policies rather than trusting the mobile UI. Supabase supports Row Level Security for controlling what authenticated users may access. ⁶

Cloudflare

I would use Cloudflare as your **front door and nervous system**, not as the database.

It gets two types of traffic:

PHONE REQUESTS

```
POST /commands
POST /tasks/:id/delegate
POST /approvals/:id
GET /projects/:id/status
```

and:

EXTERNAL EVENTS

```
POST /webhooks/github
POST /webhooks/vercel
POST /webhooks/aws
POST /webhooks/cloudflare
```

The Worker should do very little heavy reasoning:

```
Receive
  ↓
Authenticate/verify signature
  ↓
Identify team/project
  ↓
Normalize event
  ↓
Store/dedupe
  ↓
Push to queue
  ↓
Return quickly
```

That matters for GitHub specifically: GitHub recommends authenticating webhook deliveries with a secret, HTTPS, responding with a 2XX status within ten seconds, and moving longer work into an asynchronous queue. GitHub also exposes a unique delivery identifier that can be used to prevent duplicated/replayed processing. ⁷

Cloudflare Queues fits that architecture because it can offload work from requests and supports retries and buffering. Its delivery guarantee is **at least once**, meaning duplicates are possible, so your event processor should use an idempotency/event ID rather than assuming every message arrives exactly once. ⁸

Your normalized internal event might look like:

```
{
  "event_id": "github:86e192...",
  "team_id": "team_1",
  "project_id": "project_7",
  "provider": "github",
  "type": "deployment.failure",
  "severity": "critical",
  "resource": "storefront",
  "source_ref": "deployment_932",
  "occurred_at": "2026-09-12T10:30:00Z"
}
```

The huge advantage is that your AI doesn't need separate logic everywhere for:

```
Vercel failed
AWS Lambda failed
GitHub Action failed
Cloudflare Worker failed
```

Your system converts all of them into an internal vocabulary:

```
build.failed
deployment.failed
runtime.error
latency.anomaly
cost.anomaly
pull_request.opened
code.merged
task.assigned
agent.completed
```

Then the rest of your application operates on **events**, not vendor-specific payloads.

The intelligence and execution layers

This is the part that makes your idea more interesting than “GitHub on a phone.”

You need **four different kinds of intelligence/execution**, and they should not be collapsed into one giant LLM.

Local phone AI

The iQOO model handles:

```
Intent classification
Voice command understanding
Notification priority
Short summaries
Simple task routing hints
Offline command drafting
Approval explanation
Cached project Q&A
```

For example:

"Bro production is screwed, see what happened."

Local model:

```
{  
  "intent": "investigate_production",  
  "urgency": "high",  
  "needs_cloud_agent": true  
}
```

That's enough.

Don't try to feed an entire 50,000-file repository into a phone model.

The orchestrator

This should be the **actual brain of your product**.

Think of it as a deterministic state machine with AI inside it.

Suppose your request is:

"Implement dark mode."

The orchestrator creates:

```
TASK  
  objective: Add dark mode  
  project: storefront  
  repository: github/company/storefront  
  requested_by: Sid  
  risk: medium  
  status: planning
```

Then:

```
Planner  
  ↓  
Need repository access?  
  ↓ yes  
Create sandbox  
  ↓  
Need powerful reasoning?
```

```
    ↓ yes
Call cloud model
    ↓
Run tests
    ↓
Passed?
    ↓ yes
Open PR
    ↓
Preview deployment
    ↓
Verification
    ↓
Human approval
    ↓
Merge
```

Crucially, the orchestrator decides **what tools the model is allowed to call**.

The model should never be handed:

```
AWS_ROOT_KEY=...
GITHUB_ADMIN_TOKEN=...
VERCEL_TOKEN=...
```

Instead it receives tools conceptually like:

```
read_repository()
search_code()
run_tests()
write_file()
create_branch()
create_pull_request()
fetch_deployment_logs()
get_recent_deployments()
```

Then your **connector broker** executes those operations after checking permissions.

That makes the model a reasoning engine rather than a god process.

Cloud AI

For hard tasks, the orchestrator escalates to NVIDIA NIM or another large model.

NVIDIA currently describes NIM as deployable microservices for running foundation models on cloud or data-center infrastructure, making it reasonable as one inference backend for your architecture. ⁹

But don't hard-code your product around:

```
NVIDIA = intelligence
```

Do:

```
Model Router
|
├─ Small local model
├─ NIM coding model
├─ reasoning model
├─ vision model
└─ cheap summarization model
```

Then the orchestration layer can say:

```
Notification summary      → Local model
Simple code explanation    → Cheap cloud model
Complex bug               → Strong coding model
Architecture investigation → Strong reasoning model
Screenshot understanding   → Vision model
```

The product is therefore **model-independent**.

The cloud computer

This is one of the strongest parts of your idea.

The LLM should not pretend it edited a repository.

Give it a real Linux environment.

Vercel Sandbox is now explicitly built to run agent-generated/untrusted code inside isolated Firecracker microVMs with separate filesystems and networking, and it exposes programmatic APIs for creating sandboxes, running commands and managing files. ³

One agent session might become:

```
Sandbox created
↓
git clone repository
↓
checkout agent/task-842
↓
npm install
↓
AI reads repository
↓
AI edits code
↓
npm test
↓
npm run build
↓
start dev server
↓
verify page
↓
git diff
↓
git commit
↓
push branch
↓
create PR
↓
sandbox stopped
```

That is your:

mini computer sitting in the cloud.

And it means the user's own laptop doesn't even need to be online.

This is an important architectural point:

WRONG

iQ00 → personal laptop → Claude Code → GitHub

BETTER

iQOO → control plane → isolated cloud computer → GitHub

Your laptop can still be used during the hackathon for development and Office Kit interactions, but your actual product architecture becomes truly phone-first rather than “a laptop app remote-controlled from a phone.” That also aligns better with the event's phone-first format. ¹

End-to-end flows

Now let's put everything together.

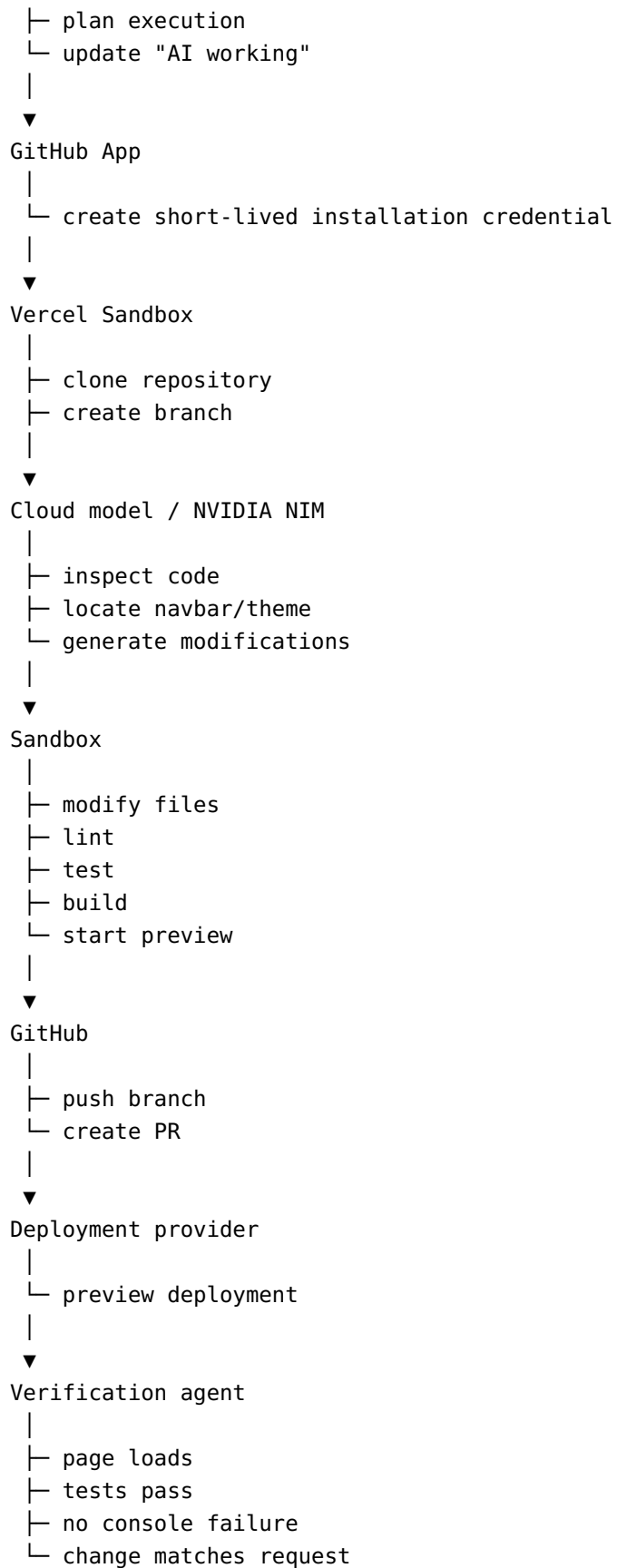
Shipping a feature from the phone

You say into iQOO:

“Add a dark-mode switch to the navbar and ship it.”

The complete system flow should be:

```
YOU
|
▼
iQOO local model
|
├─ intent = code_feature
├─ project = storefront
├─ objective = dark-mode navbar
└─ risk = code modification
|
▼
Cloudflare API
|
├─ validate auth
├─ check team membership
└─ forward command
|
▼
Supabase
|
└─ create Task #384
|
▼
Orchestrator
|
├─ determine repository
└─ determine permissions
```



```

|
▼
Supabase Realtime
|
└─ notify team
|
▼
iQ00

```

The phone then shows:

```

| DARK MODE FEATURE
|
| ✓ Code modified
| ✓ 38 tests passed
| ✓ Build passed
| ✓ Preview deployed
| ✓ Visual check passed
|
| Files changed: 4
| Risk: Medium
|
| [View Diff]
| [View Preview]
|
| [ APPROVE MERGE ]

```

GitHub's App model fits this well because installation credentials can be limited to repositories and specific permissions and expire after one hour. GitHub's API also supports GitHub App installation tokens for repository and pull-request operations when the appropriate permissions are granted. ¹⁰

When you approve:

```

BiometricPrompt
  ↓
Android Keystore signs:
{
  approval_id,
  task_id,
  commit_sha,
  action: "merge_and_deploy",
  timestamp
}

```

```

      ↓
Backend verifies signature
      ↓
Orchestrator executes approved action

```

Android Keystore can keep app cryptographic key material non-exportable, can hardware-bind keys where supported, and can require user authentication such as biometrics before a particular cryptographic operation. ¹¹

That gives you a **seriously good security story**:

AI can propose production changes, but the user's phone cryptographically authorizes dangerous actions.

Detecting a production failure

Now suppose production starts failing.

You don't want:

AI repeatedly asks all APIs every 5 seconds

You want **event-driven architecture**.

```

Application/Vercel/AWS
|
| failure / monitoring event
▼
Cloudflare webhook gateway
|
├─ verify source
├─ normalize
├─ deduplicate
└─ enqueue
▼
Incident processor
|
├─ severity?
├─ affected component?
├─ recent deployment?
└─ corresponding repository?
▼
AI triage
|
├─ fetch relevant logs

```

```
└─ recent commits
└─ deployment metadata
└─ ownership information
```



Incident created

AWS CloudWatch Logs is built for log data from AWS workloads, while AWS Cost Explorer exposes programmatic cost-and-usage data, so those can back your AWS observability and cost connectors. ¹²

Your phone might receive:



CRITICAL INCIDENT

Checkout errors increased.

First detected: 16:41

Environment: Production

Affected endpoint: /api/checkout

Likely cause:

Commit 91ef84

"refactor payment handler"

Confidence: 84%

Likely team:

Frontend / Checkout

Owner:

Sid

[I'LL HANDLE IT]

[DELEGATE TO AI]

Firebase Cloud Messaging gives you a standard Android path for server-initiated push delivery when the app needs to be notified in the background rather than relying exclusively on an open realtime connection. ¹³

Automatic ownership and team routing

This is where your idea becomes genuinely interesting.

Don't assign incidents based only on:

```
frontend error → frontend developer
```

Build an **ownership resolver**.

Inputs:

```
Explicit project role  
CODEOWNERS  
Files involved in error  
Recent commit authors  
Current task  
Current presence  
Current workload  
Availability  
Incident history
```

GitHub already has a CODEOWNERS mechanism for assigning responsibility for parts of a repository; code owners are automatically requested when pull requests change code they own. That makes CODEOWNERS a useful high-confidence input into your own task-routing algorithm. ¹⁴

A starting score could be your own heuristic such as:

```
OwnerScore =  
  35% explicit project responsibility  
+ 30% CODEOWNERS match  
+ 15% active context  
+ 10% availability  
+ 10% recent relevant commits
```

That weighting is a **product design recommendation**, not a GitHub feature.

Example:

```
ERROR  
  
src/components/CheckoutForm.tsx
```

Ownership resolver sees:

```
CODEOWNERS  
/frontend/checkout/* → @sid
```

Sid role
Frontend

Sid current task
Navbar dark mode

Sid availability
Online

Recent commits
Sid = 74% of CheckoutForm changes

Result:

Recommended owner: Sid
Confidence: 94%

Supabase gets:

```
incident.owner = sid  
incident.status = assigned
```

Supabase Presence becomes:

Sid
● Working
Incident #92
Checkout production failure

The other team members instantly see it.

Delegating that incident to AI

You tap:

Delegate to AI

Then:

Incident
↓
Orchestrator

```
↓
Collect evidence
├─ production logs
├─ error stack
├─ last successful deployment
├─ first failing deployment
├─ commit diff
└─ repository context
↓
Create isolated cloud sandbox
↓
Clone repo
↓
Reproduce issue
↓
Coding model
↓
Fix
↓
Tests
↓
Preview
↓
PR
↓
Phone approval
```

Phone status could update in realtime:

AI INVESTIGATING #92

- ✓ Retrieved production logs
- ✓ Identified suspected commit
- ✓ Repository cloned
- ✓ Error reproduced
- Testing proposed fix
 - Preview deployment
 - Human review

Meanwhile everyone's team screen says:

```
Sid
Frontend
Incident #92
Delegated → AI Coder
```

```
AI Coder
Working
Fixing CheckoutForm
```

That's **shared AI awareness**.

Not “AI chat.”

Not “an AI coding app.”

The system is now representing **both humans and software agents as participants in the same engineering organization**.

Deployment and verification

Do not let the AI immediately push to production.

Use:

```
AI branch
  ↓
Pull request
  ↓
Tests
  ↓
Preview deployment
  ↓
Automatic verification
  ↓
Human approval
  ↓
Merge
  ↓
Production deployment
  ↓
Post-deploy health window
```

GitHub supports Pull Request creation and merge operations through appropriately permissioned GitHub App tokens. ¹⁵

After deployment, your product should compare:

Before deployment

error_rate
latency
HTTP failures
health endpoint
critical user flow

versus:

After deployment

error_rate
latency
HTTP failures
health endpoint
critical user flow

Then the app can say:

DEPLOYMENT HEALTH

Version	v1.8.2
Commit	c45de8
Deploy	✓
Homepage	✓
Checkout	✓
Login	✓
Error rate	0.21%
Regression	none detected

Status: HEALTHY

For the hackathon, you don't need industrial monitoring. A scheduled HTTP health check plus one browser verification after deployment is enough to demonstrate the architecture.

Team intelligence, monitoring and integrations

Your feature list looks huge because you're thinking of every vendor separately.

Architecturally, though, most of it reduces to **three interfaces**:

```
READ
"What is happening?"

ACT
"Do something."

EVENT
"Something happened."
```

Every connector implements those three concepts.

GitHub connector

```
READ
repository
branches
commits
PRs
files
checks

ACT
create branch
push change
create PR
merge

EVENT
push
PR opened
PR merged
workflow/check state
```

A **GitHub App** is better suited than a user's permanent personal access token for your architecture because it can authenticate per installation and produce short-lived, permission-limited installation credentials. ¹⁶

Vercel connector

```
READ
deployment
build state
deployment logs
preview URL
```

ACT
trigger/redeploy/rollback where supported

EVENT
deployment status

And separately:

Vercel Sandbox

not application hosting state
but AGENT EXECUTION

That distinction matters.

Vercel Deployment
=
the user's product

Vercel Sandbox
=
your AI engineer's temporary computer

Vercel documents Sandbox as a self-contained environment for agent-generated code, development servers, tests, file operations and live previews. ³

AWS connector

Start **read-only**.

For the hackathon:

CloudWatch
→ logs

Cost Explorer
→ cost data

IAM / STS
→ temporary access

AWS `AssumeRole` sessions are temporary and configurable in duration, rather than requiring you to distribute a permanent IAM access key to the phone. ¹⁷

So users can configure an AWS role:

```
iQ00DevAgentObserverRole
```

with permissions like:

```
CloudWatch Logs: read  
Cost Explorer: read
```

Later you might have:

```
iQ00DevAgentOperatorRole
```

that can perform some approved infrastructure actions.

Do not request AdministratorAccess.

Cloudflare connector

There are really two Cloudflare usages in your design.

Your own platform:

```
Cloudflare Worker  
Cloudflare Queue  
Cloudflare Workflow  
Secrets
```

and the user's Cloudflare account:

```
DNS  
Workers  
logs  
analytics  
deployments
```

Keep those credentials completely separate.

Cloudflare provides server-side secrets for sensitive values and granular API tokens, which is preferable to putting tokens into application environment variables that may be exposed to clients. ¹⁸

Cost monitoring

Your unified cost screen could eventually look like:

TODAY

AWS	₹312
Vercel	₹83
Cloudflare	₹0
AI inference	₹174
Database	₹42
TOTAL	₹611

▲ 18% vs yesterday

But I would be precise in your pitch:

“Unified provider cost intelligence” is realistic.

“Perfect live billing for every connected cloud” is not something you should promise, because availability, granularity and delay depend on each provider's billing API.

For AWS specifically, Cost Explorer's API exposes cost-and-usage information programmatically. ¹⁹

The AI layer can then add the interesting piece:

YOU:

Why did costs increase today?

AI:

AWS increased primarily in Lambda.

AI inference usage also increased.

Likely contributor:

new /search endpoint deployed at 13:42.

Would you like me to investigate?

Now your product isn't just aggregating dashboards.

It's **reasoning across them**.

Web search

Web search belongs in the cloud orchestration layer, not inside the local phone model.

Phone
"Check if this dependency has a known issue."

↓

Local AI
research_request

↓

Orchestrator

↓

Search connector / browser

↓

Relevant official docs/issues

↓

Cloud reasoning

↓

Phone summary

This can later support your more general personal-agent ideas, but for the Developer Tools submission I would restrict it to things like:

documentation lookup
dependency research
API docs
security advisories
error research
framework changes

That keeps the story coherent.

Security and authority model

This is probably where judges will attack the concept:

“You're giving an AI control of GitHub and production. What happens when it hallucinates?”

You need a very strong answer.

Your answer should be:

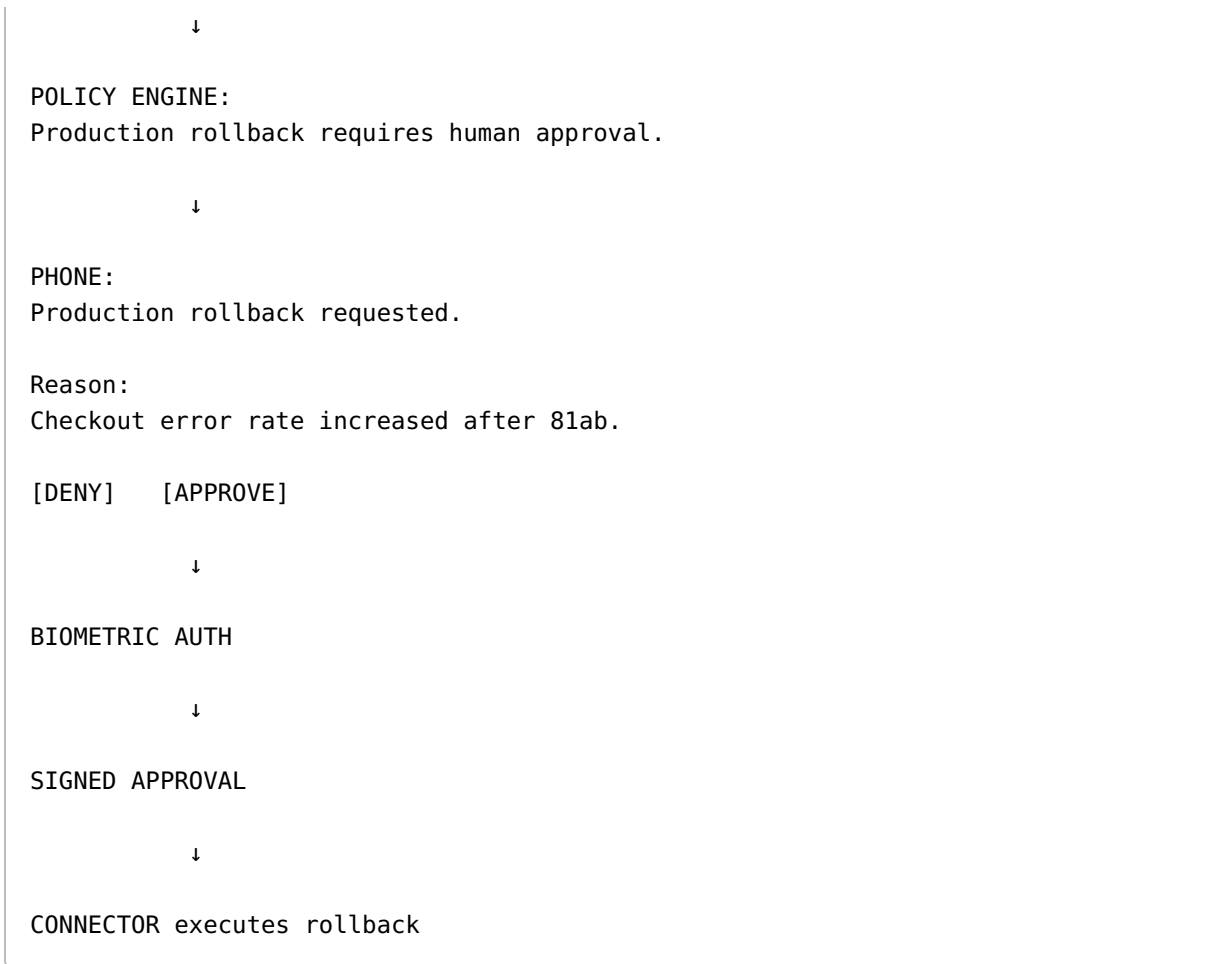
The AI does not possess authority. The policy engine does.

Design four levels.

Action	Agent allowed automatically?	Human approval
Read repository	Yes	No
Read logs	Yes	No
Analyze incident	Yes	No
Create sandbox	Yes	No
Create branch	Yes	Usually no
Modify sandbox files	Yes	No
Run tests	Yes	No
Open PR	Yes	Optional
Merge to protected branch	No	Yes
Deploy production	No	Yes
Roll back production	Policy-dependent	Usually yes
Change DNS	No	Yes
DB migration	No	Yes
Change secrets	No	Yes
Increase cloud spend	No	Yes

Then the sequence becomes:

AI:
"I think we should roll back commit 81ab."



That's a **human-in-the-loop agent platform**, not an unsupervised bot with cloud admin keys.

Credentials

Your phone should contain:

Supabase/user session
device signing key

It should **not** contain:

GitHub App private key
AWS role credentials
NVIDIA API token
Cloudflare API token
Vercel token
Supabase server secret

GitHub tokens should be minted just-in-time, scoped to the necessary installation/repositories and allowed permissions; installation tokens expire after one hour. ²⁰

AWS access should similarly prefer temporary STS sessions. ¹⁷

Server secrets should live in secret storage; Cloudflare's Worker Secrets mechanism is explicitly intended for sensitive values such as API keys and authentication tokens. ²¹

Provider tool broker

This is one of the most important pieces.

Instead of:

```
LLM
|
└─ has GitHub token
```

do:

```
LLM
|
| "create_pull_request"
▼
Tool Broker
|
├─ Is user allowed?
├─ Is agent allowed?
├─ Is repo allowed?
├─ Does action need approval?
├─ Generate temporary credential
└─ Execute GitHub call
|
▼
GitHub
```

So even a compromised model cannot simply decide:

```
rm production
```

because it doesn't possess the credentials or direct capability.

Audit log

Every meaningful action should create something like:

```
{
  "actor": "agent:coding_agent",
  "requested_by": "user:sid",
  "action": "github.pull_request.create",
  "project": "storefront",
  "resource": "PR#82",
  "approved_by": null,
  "result": "success",
  "timestamp": "..."
```

For a production merge:

```
{
  "actor": "connector:github",
  "requested_by": "agent:coding_agent",
  "approved_by": "user:sid",
  "approval_device": "iqoo-device-82",
  "commit": "8ca392",
  "action": "production.merge"
```

That audit trail also gives your AI excellent historical context.

What you should actually build for the hackathon

Your current 25 features are a **startup roadmap**, not a 30-hour hackathon MVP.

Trying to genuinely implement:

```
GitHub
Vercel
AWS
Cloudflare
Supabase
payments
costs
monitoring
web search
local AI
```

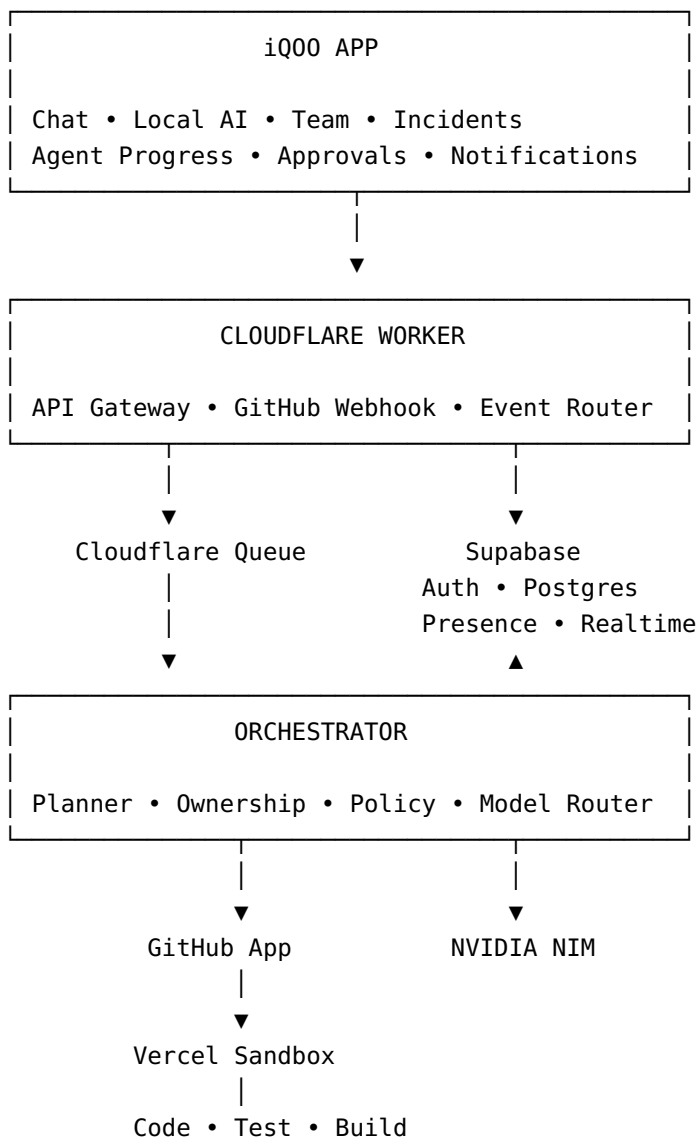
cloud AI
cloud computer
team collaboration
automatic routing
production deployment

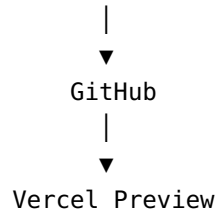
in one weekend would likely give you **25 half-working features and no killer demo.**

Instead, make the architecture broad but make the implementation narrow.

I would build this:

The core stack





That is enough.

Demo storyline

Your **first demo**:

"Ship this feature."

Judge speaks into phone:

"Change the homepage CTA from 'Get started' to 'Build with AI' and add a new card."

Phone shows:

UNDERSTOOD LOCALLY

Task:

Modify homepage CTA + card

Project:

Hackathon Demo

[Delegate]

Tap.

Then:

AI Agent

- ✓ Sandbox created
- ✓ Repository cloned
- ✓ Homepage located
- Editing

Next:

- ✓ Changes complete
- ✓ Tests passed
- ✓ Build passed
- ✓ PR #42 created
- ✓ Preview ready

Phone:

VIEW PREVIEW

VIEW DIFF

[APPROVE]

Biometric confirmation.

Production updates.

That demonstrates:

AI Coding Agent + GitHub + cloud computer + local AI + phone control + deploy + approval.

Demo storyline for the team feature

Then deliberately break your demo website or simulate a known error.

GitHub/deployment event arrives.

Your system says:



PRODUCTION INCIDENT

Homepage crashed after deployment.

Component:

PricingCards.tsx

Type:

Frontend

Likely owner:

Sid

Reason:

Frontend role + repository ownership

[TAKE INCIDENT]

[DELEGATE TO AI]

On teammate's iQOO:

TEAM ACTIVITY

Sid

Incident #81

Investigating frontend failure

Arjun

Backend

Available

AI Coder

Waiting

You tap:

Delegate to AI

Then:

Sid

Incident #81

Delegated to AI

AI Coder

● Working

Agent fixes it in Sandbox, opens PR and presents approval.

That second demo demonstrates:

Monitoring + automatic ownership + realtime teams + human/AI delegation + notifications + coding agent + preview deployment.

Put those two demos together and you've essentially demonstrated the heart of nearly all 25 features.

Build these for real

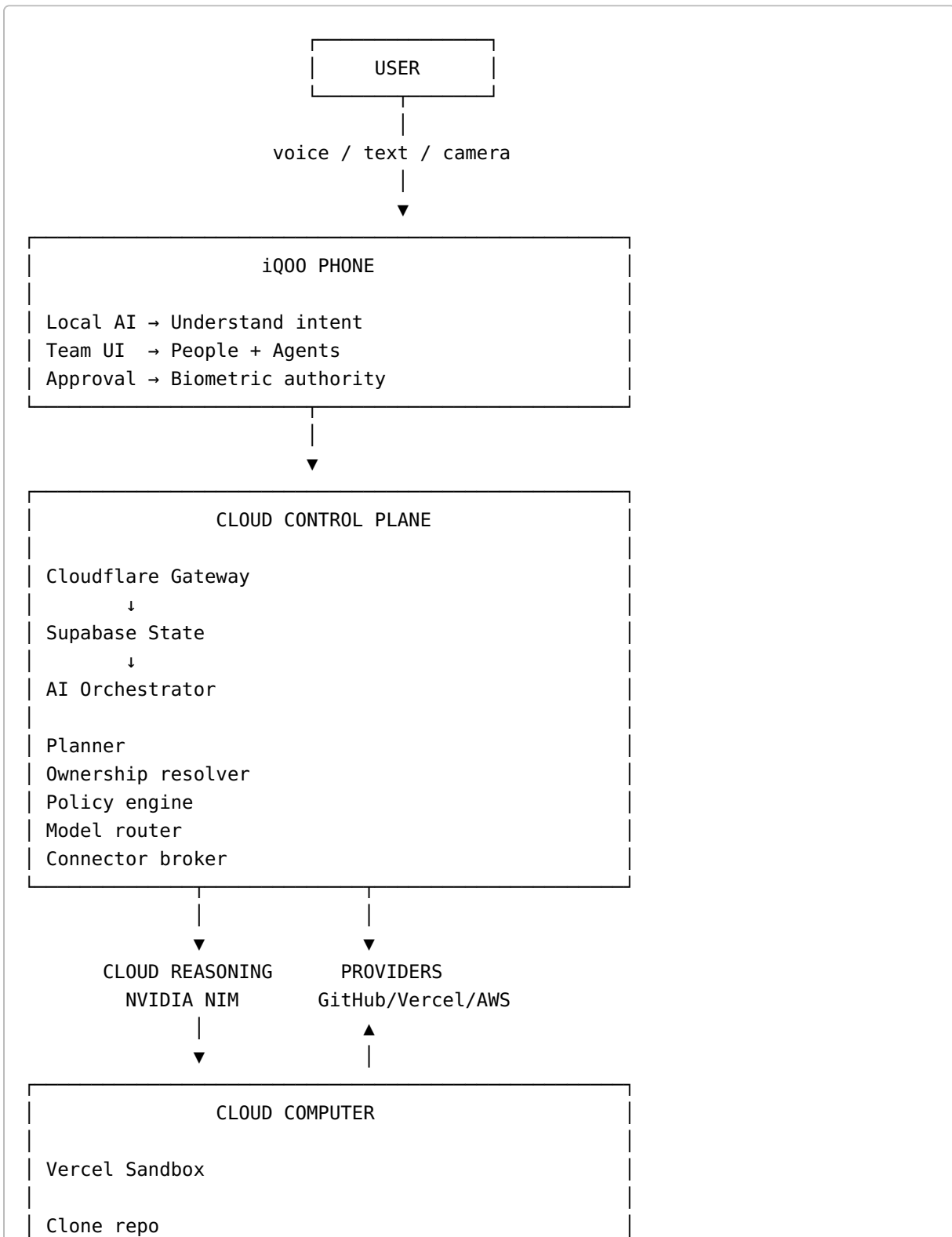
For the hackathon, I would genuinely implement:

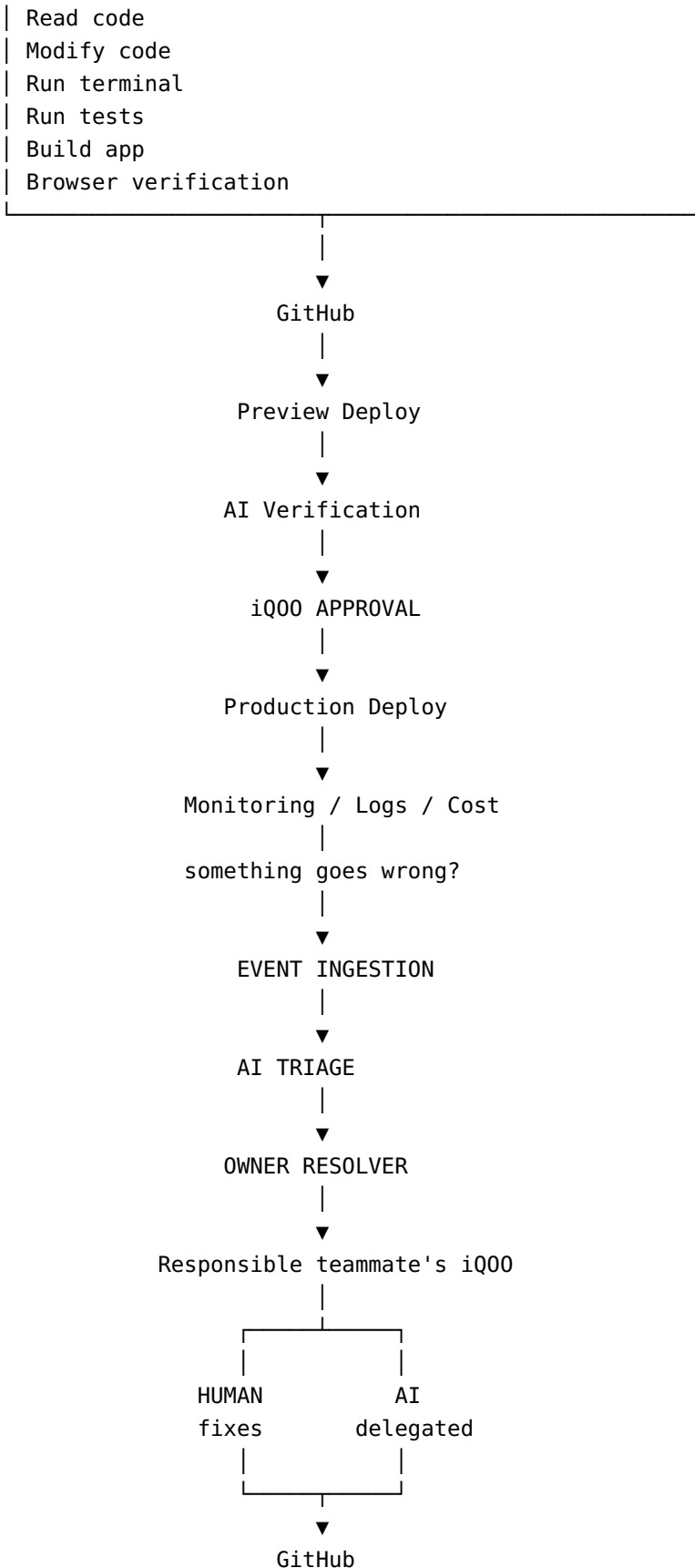
Capability	Build status
iQOO Android app	Real
Small on-device model	Real
Supabase Auth	Real
Team accounts	Real
Team roles	Real
Supabase Presence	Real
Task status	Real
GitHub App	Real
GitHub webhook	Real
Cloudflare Worker	Real
One queue/event processor	Real
Cloud model/NIM	Real
Cloud sandbox	Real
Agent edits repository	Real
Tests/build	Real
PR creation	Real
Vercel preview	Real
Phone approval	Real
One monitoring/error flow	Real
AI owner assignment	Real but simple
AWS integration	Stretch
AWS cost monitoring	Stretch
Cloudflare-user infrastructure control	Stretch
Multi-provider billing	Roadmap
Full autonomous DevOps	Roadmap

That way you have a **real vertical slice**, not mock screens.

Your complete product loop

The final architecture can be summarized in one loop:







That is the product.

And I would describe it to the iQOO judges in one sentence:

"We turn the iQOO phone into the secure AI control plane for an entire software team: it understands what is happening across code, deployments, infrastructure and people; detects problems, finds the right owner, coordinates human and AI engineers, gives agents isolated cloud computers to make and test changes, and brings production decisions back to the phone for human approval."

That is substantially stronger than **"AI that codes from your phone."**

The deeper product idea is:

**GitHub knows the code.
Vercel knows deployments.
AWS knows infrastructure.
Supabase knows your team.
Your AI layer knows the relationship between all of them.**

And the iQOO phone becomes the place where that entire engineering system is observed, commanded and authorized.

1 Guide & Rules · iQOO Hackathon 2026 · City Battles

https://iqoo.reskilll.com/?utm_source=chatgpt.com

2 10 16 20 Authenticating as a GitHub App installation - GitHub Docs

<https://docs.github.com/en/apps/creating-github-apps/authenticating-with-a-github-app/authenticating-as-a-github-app-installation>

3 Vercel Sandbox

<https://vercel.com/docs/vercel-sandbox>

4 LiteRT for Android | Google AI Edge | Google for Developers

<https://ai.google.dev/edge/litert/android>

5 Presence | Supabase Docs

<https://supabase.com/docs/guides/realtime/presence>

6 Row Level Security | Supabase Docs

<https://supabase.com/docs/guides/database/postgres/row-level-security>

7 Best practices for using webhooks - GitHub Docs

<https://docs.github.com/en/webhooks/using-webhooks/best-practices-for-using-webhooks>

8 Overview · Cloudflare Queues docs

<https://developers.cloudflare.com/queues/>

9 API Key Authentication for RAG and Retriever Evaluations

https://docs.nvidia.com/?utm_source=chatgpt.com

11 Android Keystore system | Security | Android Developers

<https://developer.android.com/privacy-and-security/keystore>

12 What is Amazon CloudWatch Logs? - Amazon CloudWatch Logs

<https://docs.aws.amazon.com/AmazonCloudWatch/latest/logs/WhatIsCloudWatchLogs.html>

13 Receive messages in Android apps | Firebase Cloud Messaging

<https://firebase.google.com/docs/cloud-messaging/android/receive>

14 About code owners - GitHub Docs

https://docs.github.com/repositories/managing-your-repositorys-settings-and-features/customizing-your-repository/about-code-owners?utm_source=chatgpt.com

15 REST API endpoints for pull requests - GitHub Docs

https://docs.github.com/en/rest/pulls/pulls?apiVersion=2026-03-10&utm_source=chatgpt.com

17 AssumeRole - AWS Security Token Service

https://docs.aws.amazon.com/STS/latest/APIReference/API_AssumeRole.html

18 21 Secrets · Cloudflare Workers docs

<https://developers.cloudflare.com/workers/configuration/secrets/>

19 GetCostAndUsage - AWS Billing and Cost Management

https://docs.aws.amazon.com/aws-cost-management/latest/APIReference/API_GetCostAndUsage.html